Department of Computer Science and Engineering

Course Code & Name	CSA06 – Design and Analysis of Algorithms
Assessment	CO2 – Debugging and Optimisation Task
Course Outcome Assessed	Apply Brute Force technique to solve sorting, searching, Closest-Pair and Convex-Hull problems.
Maximum Marks	**50**
Duration	1.5 hrs (Assessment conducted in class)
Mode	Individual
Student Reg. No.	192525458
Student Name	S.Sharvesh

Instructions: Debug the code by reasoning and testing, not by rewriting it from scratch. For every change, state what was wrong, why it was wrong and how you verified the fix. Optimisations must preserve the corrected outputs. Marks for each part are shown in brackets.

DEBUGGING AND OPTIMISATION TASK (50 MARKS)

Scenario – SOC Alert-Triage Module of "SentinelOps": SentinelOps is the Security Operations Centre (SOC) of a cloud-services company. A junior developer wrote the Python module below to (1) sort incoming alerts by severity, highest first, keeping alerts of equal severity in arrival order; (2) check whether a source IP address is on the blocklist; (3) find every position of an attack signature in a log line; and (4) find the two login attempts whose locations are closest to each other, which the SOC uses to flag clustered credential-stuffing attacks. The module passed code review but failed in production: alerts disappeared or were duplicated, blocked IP addresses got through, signatures at the end of a log were missed while false signatures were reported, and the "closest logins" report flagged logins hundreds of kilometres apart. The module must be fixed before it processes about 50,000 alerts and batches of 8,000 login events per day.

Code under investigation:

```
import math

def sort_alerts(alerts):            # insertion sort by severity, highest first
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts

def is_blocked(blocklist, ip):      # sequential search
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):   # brute-force string matching, all occurrences
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
    return hits

def closest_logins(events):         # brute-force closest pair of login locations
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

Test data:

```
alerts    = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]
queries   = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]
log, pattern = "FAILFAIXFAIL", "FAIL"
events    = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]   # (x, y) in
km
```

(a) Reproduce and Identify the Defects (10 Marks): Run the module on the test data. For each function, record the expected output and the actual output, then locate every defect: state the function, the faulty line and the nature of the error. The module contains nine defects, and none of them is marked in the code.

(b) Explain the Root Causes and Debug Systematically (12 Marks): For each defect, explain its root cause and how it produces the observed wrong output. Correct the functions one at a time, supporting your reasoning with a step-by-step trace (for example, the insertion trace for the alerts list and the alignment table for the log), and re-test after every correction. Include boundary tests such as an empty blocklist, a pattern at the very end of the log, a pattern longer than the log, equal severities and fewer than two login events.

(c) Optimise the Corrected Module (10 Marks): Without changing any output, improve the corrected module for the production workload. Justify whether insertion sort remains suitable for nearly sorted alerts; propose a better approach for repeated blocklist lookups; make sure the signature scan performs no redundant comparisons; and, for closest_logins, eliminate self and

repeated pairs, remove unnecessary square-root computations and prune pairs that cannot be closest.

(d) Analyse Before and After (10 Marks): Derive the number of basic operations and the time complexity of each function before and after optimisation. Measure the distance computations of the brute-force and optimised closest_logins for at least three batch sizes between 1,000 and 8,000 events, and compare insertion-sort comparisons on nearly sorted and random alerts. Explain whether the measurements agree with your analysis.

(e) Deliver Production-Quality Code (8 Marks): Submit the final, optimised module with docstrings, meaningful comments and a test harness that checks every function, including the boundary cases from (b). Commit the code to your GitHub repository and share the link through Google Classroom.

Rubrics – Debugging and Optimisation Task

Criteria	Weightage (%)	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Identification	20	Accurately identifies all errors and root causes with clear understanding	Identifies most errors with minor gaps	Identifies some errors but misses key issues	Unable to identify errors correctly
Debugging	25	Applies systematic debugging techniques effectively to resolve issues	Uses appropriate debugging methods with minor errors	Limited debugging approach; requires guidance	Unable to debug or resolve issues
Optimization	20	Optimizes code by significantly improving time complexity using efficient algorithms	Improves time complexity with minor gaps in efficiency	Limited improvement in time complexity	No improvement; inefficient approach retained
Analysis	20	Demonstrates strong logical reasoning and clear justification of solutions	Shows reasonable analysis with some justification	Limited analysis; weak reasoning	No analytical reasoning demonstrated
Code Quality	15	Produces clean, well-structured code with proper comments and documentation	Code is mostly clear with minor documentation gaps	Code lacks structure and proper documentation	Poorly written code with no documentation

Code of Conduct

I S.Sharvesh(192525458)_____ (Name / Reg. No.) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S.Sharvesh_____ Date: 19-10-2026_____

(a) REPRODUCE AND IDENTIFY THE DEFECTS

The given Python module contains four functions: insertion sort, sequential search, brute-force string matching and closest-pair search. Testing the supplied data reveals the following defects.

1. Sorting – Line 40: The condition j > 0 should be j >= 0. Otherwise, the first element is not considered for shifting.

2. Sorting – Line 43: The insertion statement alerts[j] = key should be alerts[j + 1] = key. When j becomes -1, the key is inserted at the wrong position.

3. Blocklist – Line 49: The function returns False when an IP address matches. It should return True.

4. Blocklist – Line 50: The else statement returns False immediately after the first nonmatching address. The search should continue through the entire blocklist.

5. Signature matching – Line 56: range(n - m) excludes the final valid starting position. It should be range(n - m + 1).

6. Signature matching – Line 60: Checking j == m - 1 can report a false match when the last character mismatches. The correct condition is j == m.

7. Closest pair – Line 65: Initialising min_dist to zero prevents the algorithm from correctly identifying the minimum distance. Initialise the squared distance to infinity.

8. Closest pair – Line 71: The comparison d > min_dist finds the farthest pair. It must compare against the current minimum.

9. Closest pair – Lines 67–68: The loops include self-pairs and repeated pairs. Use j = i + 1 to evaluate each unique pair once.

Expected and actual results:

1. sort_alerts() Expected: [('A2', 9), ('A4', 9), ('A3', 5), ('A1', 3), ('A5', 1)]

The original implementation produces an incorrect list with duplicated and missing alerts because of incorrect shifting and insertion positions.

2. is_blocked() Queries: 10.0.0.9, 10.0.0.5, 8.8.8.8 Expected: True, True, False Actual: False, False, False

3. find_signature() Log: FAILFAIXFAIL Pattern: FAIL Expected: [0, 8] Actual: [0, 4]

4. closest_logins() Expected: Pair (0, 5), distance $\sqrt{2} \approx 1.4142$ km. The original implementation returns the farthest pair, (2, 3), with distance $\sqrt{3125} \approx 55.90$ km.

(b) EXPLAIN ROOT CAUSES AND DEBUG SYSTEMATICALLY

1. Debugging insertion sort

Corrected code:

def sort_alerts(alerts): for i in range(1, len(alerts)): key = alerts[i] j = i - 1 while j >= 0 and alerts[j][1] < key[1]: alerts[j + 1] = alerts[j] j -= 1 alerts[j + 1] = key return alerts

Trace:

Initial: A1(3), A2(9), A3(5), A4(9), A5(1) Pass 1: A2(9), A1(3), A3(5), A4(9), A5(1) Pass 2: A2(9), A3(5), A1(3), A4(9), A5(1) Pass 3: A2(9), A4(9), A3(5), A1(3), A5(1) Pass 4: A2(9), A4(9), A3(5), A1(3), A5(1)

The corrected function sorts alerts in descending severity order. The strict comparison preserves the

arrival order of alerts with equal severity.

2. Debugging sequential search

Corrected code:

def is_blocked(blocklist, ip): for address in blocklist: if address == ip: return True return False

The original code returns False for matching addresses and stops after the first nonmatch. The corrected code returns True when a match is found and False only after the complete list has been checked.

Verification: 10.0.0.9 → True 10.0.0.5 → True 8.8.8.8 → False Empty blocklist → False

3. Debugging string matching

Corrected brute-force code:

def find_signature(log, pattern): n, m = len(log), len(pattern) hits = []

```
if m == 0:
    return list(range(n + 1))

for i in range(n - m + 1):
    j = 0
    while j < m and log[i + j] == pattern[j]:
        j += 1
    if j == m:
        hits.append(i)

return hits
```

Alignment: Index 0: FAIL → Match Index 1: AILF → Mismatch Index 2: ILFA → Mismatch Index 3: LFAI → Mismatch Index 4: FAIX → Mismatch Index 5: AIXF → Mismatch Index 6: IXFA → Mismatch Index 7: XFAI → Mismatch Index 8: FAIL → Match

Expected output: [0, 8]

The corrected loop includes the final valid alignment and confirms that every character matches before recording a position.

4. Debugging closest-pair search

Corrected code:

import math

def closest_logins(events): if len(events) < 2: return None, None

```
min_dist_sq = float("inf")
pair = None

for i in range(len(events)):
    for j in range(i + 1, len(events)):
        dx = events[i][0] - events[j][0]
        dy = events[i][1] - events[j][1]
        d_sq = dx * dx + dy * dy

        if d_sq < min_dist_sq:
            min_dist_sq = d_sq
            pair = (i, j)
```

```
return pair, math.sqrt(min_dist_sq)
```

The original function finds the maximum distance because it uses an incorrect initial value and reversed comparison. The corrected code compares each unique pair and retains the smallest squared distance.

For events (2, 3) and (3, 4):

Distance = $\sqrt{[(3 - 2)^2 + (4 - 3)^2]} = \sqrt{2} \approx 1.4142$ km

Expected result: ((0, 5), 1.4142135623730951)

Boundary tests:

- Empty blocklist returns False.
- Pattern at the end of a log is detected.
- Pattern longer than the log returns [].
- Empty pattern returns every valid boundary.
- Equal severities preserve arrival order.
- Fewer than two login events return (None, None).

(c) OPTIMISE THE CORRECTED MODULE

1. Alert sorting

Insertion sort has O(n) best-case complexity for already sorted input and $O(n^2)$ worst-case complexity. For 50,000 alerts, the worst-case comparison count is:

$n(n - 1)/2 = 50,000 \times 49,999 / 2 = 1,249,975,000$

For general production data, Python's stable sorting algorithm is preferable.

```
def sort_alerts(alerts): return sorted(alerts, key=lambda alert: -alert[1])
```

This preserves equal-severity arrival order and has O(n log n) worst-case time complexity.

2. Blocklist optimisation

Convert the blocklist to a set once:

```
blocked_ips = set(blocklist)
```

Check membership using:

```
blocked = ip in blocked_ips
```

Set construction takes O(b) expected time. Subsequent membership checks take average O(1), compared with O(b) for sequential search.

3. Signature matching optimisation

The Knuth-Morris-Pratt (KMP) algorithm preprocesses the pattern using a longest-prefix-suffix table. It avoids redundant comparisons and finds all occurrences in O(n + m) time, including overlapping matches.

4. Closest-pair optimisation

Sort the points by x-coordinate. Compare only later points, use squared distances and stop when the squared horizontal separation exceeds the best known squared distance.

This removes self-pairs, repeated pairs and unnecessary square-root operations inside the loop.

Sorting costs O(n log n), but the pruned search still has $O(n^2)$ worst-case complexity.

A divide-and-conquer closest-pair algorithm can achieve O(n log n) worst-case complexity.

(d) ANALYSE BEFORE AND AFTER OPTIMISATION

Let a be the number of alerts, b the blocklist size, n the log length, m the pattern length and k the number of login events.

1. Sorting: Insertion sort: $O(a^2)$ worst case. Python stable sorting: O(a log a) worst case.
2. Blocklist: Sequential search: O(b) per query. Set lookup: O(1) average per query.
3. Signature matching: Brute force: O(nm) worst case. KMP: O(n + m).
4. Closest pair: Brute force: $O(k^2)$. Pruned search: $O(k^2)$ worst case. Divide-and-conquer alternative: O(k log k).

Unique pair calculations:

For k = 1,000: 1,000 × 999 / 2 = 499,500 pairs.

For k = 4,000: 4,000 × 3,999 / 2 = 7,998,000 pairs.

For k = 8,000: 8,000 × 7,999 / 2 = 31,996,000 pairs.

The pruned algorithm's actual number of distance calculations depends on the coordinates and must be measured experimentally.

Benchmark procedure:

- Generate login batches of 1,000, 4,000 and 8,000 events.
- Measure brute-force and pruned distance calculations and execution times.
- Generate nearly sorted and random alert lists.
- Count insertion-sort comparisons for both inputs.
- Record actual results from the target computer.

Expected analysis: Brute-force pair calculations grow quadratically. Pruning may reduce actual comparisons but does not eliminate the quadratic worst case. Insertion sort performs close to linear work on nearly sorted data but quadratic work on random data. Python's stable sort is preferable for general workloads.

(e) DELIVER PRODUCTION-QUALITY CODE

The following final module integrates stable sorting, set membership, KMP string matching and a pruned closest-pair search.

import math

def sort_alerts(alerts): """Sort alerts by descending severity, preserving arrival order.""" return sorted(alerts, key=lambda alert: -alert[1])

def is_blocked(blocklist, ip): """Check whether an IP address belongs to a blocklist.""" return ip in blocklist

def find_signature(log, pattern): """Return all pattern positions using KMP.""" n, m = len(log), len(pattern)

```
if m == 0:
    return list(range(n + 1))
if m > n:
    return []
```

```
lps = [0] * m
length = 0
i = 1

while i < m:
    if pattern[i] == pattern[length]:
        length += 1
        lps[i] = length
        i += 1
    elif length:
        length = lps[length - 1]
    else:
        i += 1

hits = []
i = j = 0

while i < n:
    if log[i] == pattern[j]:
        i += 1
        j += 1
        if j == m:
            hits.append(i - j)
            j = lps[j - 1]
    elif j:
        j = lps[j - 1]
    else:
        i += 1

return hits
```

def closest_logins(events): """Return original indices and distance of the closest pair.""" n = len(events) if n < 2: return None, None

```
points = sorted(
    (x, y, index)
    for index, (x, y) in enumerate(events)
)

best_sq = float("inf")
best_pair = None

for i in range(n - 1):
    x1, y1, idx1 = points[i]

    for j in range(i + 1, n):
        x2, y2, idx2 = points[j]
        dx = x2 - x1

        if dx * dx > best_sq:
            break

        dy = y2 - y1
        d_sq = dx * dx + dy * dy
        pair = tuple(sorted((idx1, idx2)))

        if (d_sq, pair) < (
            best_sq,
            best_pair if best_pair is not None else (n, n)
        ):
```

```
            best_sq = d_sq
            best_pair = pair

return best_pair, math.sqrt(best_sq)


def run_tests(): """Test all functions and important boundary cases.""" alerts = [ ("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1) ] blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"] events = [ (2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4) ]

assert sort_alerts(alerts) == [
    ("A2", 9), ("A4", 9), ("A3", 5),
    ("A1", 3), ("A5", 1)
]
assert sort_alerts([]) == []
assert sort_alerts([("A", 5), ("B", 5)]) == [
    ("A", 5), ("B", 5)
]

assert is_blocked(blocklist, "10.0.0.9") is True
assert is_blocked(blocklist, "10.0.0.5") is True
assert is_blocked(blocklist, "8.8.8.8") is False
assert is_blocked([], "10.0.0.9") is False

assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]
assert find_signature("ABC", "BC") == [1]
assert find_signature("ABC", "ABCDE") == []
assert find_signature("AAAA", "AA") == [0, 1, 2]
assert find_signature("ABC", "") == [0, 1, 2, 3]
assert find_signature("", "A") == []

pair, distance = closest_logins(events)
assert pair == (0, 5)
assert math.isclose(distance, math.sqrt(2))
assert closest_logins([]) == (None, None)
assert closest_logins([(1, 2)]) == (None, None)
assert closest_logins([(0, 0), (3, 4)]) == ((0, 1), 5.0)

assert closest_logins([
    (0, 0), (1, 0), (10, 0), (11, 0)
])[0] == (0, 1)

print("All tests passed successfully.")
```

if **name** == "**main**": run_tests()

EXPECTED OUTPUT:

All tests passed successfully.

CONCLUSION

The SentinelOps module contained defects in insertion sort, sequential search, string matching and closest-pair detection. The defects were corrected through systematic tracing and boundary testing.

The optimised implementation uses stable sorting, set-based blocklist lookup, KMP pattern matching and a pruned closest-pair search. Complexity analysis demonstrates improvements in sorting, repeated lookups and signature matching. The test harness verifies normal inputs and important boundary cases.

Before submission, execute the code, record actual benchmark measurements, and commit your

verified Python file to GitHub. Submit the repository link through Google Classroom.